

문제 1. Docker Hub Image Pull Rate Limit

전제

Docker Hub usage and rate limits — 6시간 기준

계정	한도
Business / Team / Pro (인증)	Unlimited
Personal (인증)	200
미인증	100 per IPv4 address or IPv6 /64 subnet

미인증은 IP 단위, 인증은 계정 단위. 공유 IP가 문제가 되는 건 미인증일 때다.

Actions limits — GHA 적용 조건

조건	적용
GitHub-hosted + public 이미지	"Docker Hub's rate limit is not applied."
GitHub-hosted + private 이미지	적용됨
self-hosted (public/private 무관)	"always subject to the rate limit"

→ GitHub-hosted 러너가 public 이미지를 당기는 한 한도에 걸리지 않는다. 실제로 걸리는 곳은 self-hosted 러너, CodeBuild, EKS 노드처럼 공유 IP로 미인증 pull을 하는 쪽이다. Docker 공식 문서도 AWS CodeBuild·CircleCI·GitLab 등을 지목해 인증을 권한다.

카운팅: version check + download를 합쳐 1건, multi-arch는 아키텍처당 1 pull.

해결 방법

① docker login 추가 — 가장 싸다

미인증 pull 지점에 Secrets Manager 등으로 자격증명을 넣는다. IP 단위(100) → 계정 단위(200 또는 무제한)로 이동.

- 비용 0, 작업량 최소
- Personal 계정이면 200/6h를 전 파이프라인이 공유하므로 근본 해결은 아님
- 잔여량은 `/v2/ratelimitpreview/test/manifests/latest`에 HEAD를 날려 `ratelimit-remaining` 헤더로 확인

② 유료 플랜 (Team 이상) — 즉효

인증 상태에서 무제한. ①과 조합하면 그것만으로 끝난다.

- 구독 비용 발생
- Docker Hub 장애 시에는 여전히 빌드가 멈춘다

③ ECR Pull-Through Cache — 근본 대책

Sync an upstream registry with an Amazon ECR private registry.

```
resource "aws_ecr_pull_through_cache_rule" "dockerhub" {
  ecr_repository_prefix = "docker-hub"
  upstream_registry_url = "registry-1.docker.io"
  credential_arn        = aws_secretsmanager_secret.dockerhub.arn
}
```

이미지 참조를 `<acct>.dkr.ecr.<region>.amazonaws.com/docker-hub/library/node:22-alpine` 형태로 바꾼다.

공식 문서가 명시한 특성·제약:

- 캐시 히트 시 upstream 미접촉. **24시간 지난 뒤 pull할 때만** upstream 확인·갱신
- **upstream 갱신 실패해도 마지막 캐시본을 서빙** → Docker Hub 장애에도 빌드·배포 지속
- Docker Hub는 인증 필요 upstream — Secrets Manager 시크릿 이름이 반드시 `ecr-pullthroughcache/` **prefix**, PTC rule과 동일 계정·리전
- 태그 **immutability**를 켜면 캐시 갱신이 막힌다. **MUTABLE** 유지
- **AWS Lambda는 PTC 경유 pull 미지원**
- pull 주체에 `ecr:BatchImportUpstreamImage`, `ecr:CreateRepository` 필요
- PrivateLink만 있는 subnet은 **최초 pull에 인터넷 경로 필요**
- `aws_ecr_repository_creation_template` 으로 prefix 단위 lifecycle policy 등 기본값 지정

④ containerd registry mirror — 노드 레벨

노드 containerd의 `docker.io` 미러를 ECR PTC로 지정. **매니페스트를 안 고쳐도 된다.**

- ECR은 인증이 필요해 `ecr-credential-provider` 조합 필요
- 실패 시 원인 추적이 어렵다

- 매니페스트의 이미지 참조를 직접 ECR로 바꾸는 편이 단순하고 명시적

⑤ 자체 레지스트리(Harbor 등) — 이 문제만으로는 과하다

proxy cache 기능은 ③과 동일하다. Harbor를 쓰는 실제 이유는 **CVE 기반 pull 차단**, **세밀한 RBAC**, **온프레임·멀티클라우드 이식성**이다.

- HA Postgres·Redis·오브젝트 스토리지·TLS·업그레이드 운영 부담
- 이미지 pull 경로에 자체 운영 컴포넌트가 들어간다 — 죽으면 노드가 이미지를 못 당김
- 이미지 거버넌스가 독립 안건으로 올라올 때 재검토

정리

① 인증 으로 급한 불을 끄고, ③ ECR PTC 로 근본 해결. ② 는 계정 등급에 따라 ①만으로 끝날 수도 있으니 현재 플랜 확인이 선행.

문제 2. VPC 내부 Terraform plan/apply를 GHA로

전제

VPC 안이어야 하는 건 **provider가 사실 엔드포인트에 붙는 스택**뿐이다. AWS API와 S3 state만 쓰는 스택은 GitHub-hosted로 충분하다.

대표 사례:

- `kubernetes` / `helm` provider → EKS API가 private-only이거나 CIDR allowlist인 경우
- `opensearch` / `mysql` / `postgresql` 등 → internal ALB·private 서브넷 엔드포인트

중요: 이런 provider는 `apply` 뿐 아니라 `plan` 에서도 리소스를 refresh하므로 `plan`부터 막힌다.

네트워크만으로는 부족하다. EKS의 경우 `aws` CLI(exec auth) + **EKS access entry / aws-auth 매핑**이 있어야 하고, 없으면 401.

해결 방법

① self-hosted EC2 러너 (VPC 내 상주)

VPC 안에 EC2 한 대를 두고 GitHub Actions runner를 등록한다.

- **가장 단순.** public subnet + EIP면 NAT 없이도 인터넷이 나간다
- 러너를 어느 VPC에 둘지가 도달 범위를 결정한다. **peering은 transitive하지 않으므로** 허브 VPC에 두어야 여러 스포크에 닿는다
- 상주 admin 권한 박스가 생긴다 — Security hardening은 self-hosted가 *"can be persistently compromised by untrusted code in a workflow"* 라고 경고
- **동시 job 1개.** *"GitHub only assigns one job to a runner"* (Autoscaling 문서). 큐 대기 24시간 초과 시 자동 취소
- OS 패치·디스크 관리 필요

완화: `--ephemeral` 등록, 인바운드 SG 없이 SSM 전용, plan은 read-only 롤 / apply만 admin 롤로 분리

② CodeBuild-hosted GHA 러너 (VPC attach) — 상주 없이

Self-hosted GitHub Actions runners in AWS CodeBuild

"For each job in the workflow, CodeBuild starts a build to run an ephemeral GitHub Actions runner... Once the job is completed, the runner and the associated build process will be immediately terminated."

```
runs-on: codebuild-<project>-${{ github.run_id }}-${{ github.run_attempt }}
```

```
resource "aws_codebuild_webhook" "runner" {
  project_name = aws_codebuild_project.runner.name
  build_type   = "BUILD"
  filter_group { filter { type = "EVENT"; pattern = "WORKFLOW_JOB_QUEUED" } }
}
```

- **상주 인스턴스 없음**, 유휴 비용 0, OS 관리 불필요
- 동시성이 계정 CodeBuild 쿼터까지 자동 확장 (quotas — compute type별·리전별, 전부 조정 가능)
- **Lambda compute는 VPC 미지원** → EC2 compute 고정
- **VPC ENI는 public IP를 받지 않는다.** private subnet + NAT 또는 VPC endpoint가 필요

- 라벨로 image·instance-size·fleet override 가능 → matrix job마다 다른 스펙 사용 가능
- 앱 빌드용 CodeBuild와 동시 빌드 쿼터를 공유

runner 프로젝트 생성해줘야하고, buildspec 작성해야함

③ ARC (Actions Runner Controller) on EKS

GitHub 공식 — runner scale set으로 ephemeral 러너를 오토스케일. GitHub이 *"the recommended Kubernetes-based solution for autoscaling self-hosted runners"* 로 명시.

- 동시성·정리 문제가 가장 깔끔
- 러너가 terraform이 관리하는 클러스터 안에 살면 순환 의존이 생긴다. 별도 클러스터를 두거나 부트스트랩 경로를 따로 확보해야 함
- ARC 자체 운영 부담(컨트롤러 업그레이드, GitHub App 자격증명)

④ VPN / Tailscale 터널 — GitHub-hosted 유지

GitHub-hosted 러너에서 job 시작 시 tailnet 등에 ephemeral 노드로 조인해 사설 IP에 접근.

- VPC 안에 상주 리소스를 안 만들어도 된다
- 외부 SaaS 의존 + ACL 관리가 새로 생긴다. 자체 VPN이 이미 있으면 중복
- subnet router를 VPC에 띄워야 하므로 결국 상주 컴포넌트가 하나 생긴다

⑤ 엔드포인트를 공개 + CIDR allowlist — 비권장

EKS API를 public으로 열고 GitHub Actions IP를 허용. GitHub은 meta API로 IP 대역을 제공하지만 **대역이 넓고 수시로 바뀐다**. 사실상 전 세계 GHA 사용자에게 여는 것과 비슷해진다. internal ALB에는 적용도 안 된다.

두 문제의 연결

VPC 내부 self-hosted 러너 도입

- GitHub의 Docker Hub 면제에서 이탈 (공식 문서 명시)
- 러너 egress가 VPC NAT 단일 EIP
- 같은 NAT를 쓰는 EKS 노드 등과 100 pulls/6h 버킷 공유
- 문제 1의 ③(ECR PTC)이 선행되면 원천 차단

Terraform CI만 돌리는 러너는 컨테이너를 안 쓰므로 당장 pull이 0이다. 러너에 lint/test 컨테이너를 붙이는 시점부터 문제 1이 활성화된다.

정리

- 상주를 감수하고 가장 빨리 붙이려면 ①
 - 상주 없이 가려면 ② (NAT 필요)
 - 이미 EKS 운영이 익숙하고 CI 물량이 많으면 ③
 - VPC에 아무것도 안 만들고 싶으면 ④
 - ⑤는 피한다
-

출처

GitHub

- [Actions limits](#) — 사용 한도, Docker Hub rate limit 적용 조건
- [Autoscaling with self-hosted runners](#) — 러너당 job 1개, ephemeral 권장
- [Actions Runner Controller](#)
- [Security hardening for GitHub Actions](#)

Docker

- [Usage and rate limits](#) — 계정 등급별 한도
- [Pulls](#) — 카운팅 규칙, CI 공유 IP 경고

AWS

- [ECR pull through cache](#)
- [CodeBuild-hosted GitHub Actions runner](#)
- [CodeBuild quotas](#)